

WRITE AN AI TO PREDICT THE PERSON GENRE PREFERENCES BASES ON THE PERSON AGE AND GENDER

Requirements :

Python Libraries: pandas scikit-learn

Files: music.csv (Dataset file containing features and genre labels)

Code:

```
from sklearn.tree import DecisionTreeClassifier
import pandas as pd

music_data = pd.read_csv('music.csv')

X = music_data.drop(columns=['genre'])
y = music_data['genre']

model = DecisionTreeClassifier()
model.fit(X, y)

sample = pd.DataFrame({
    'age': [21, 22],
    'gender': [1, 0]
})
predictions = loaded_model.predict(sample)
print(f"\n\n{predictions}\n\n")
```

Output:

```
[ 'Pop'  'Dance' ]
```

Used music.csv

age,gender,genre

18,1,Hiphop

18,0,Classic

20,1,Pop

20,0,Pop

22,1,Dance

22,0,Dance

24,1,Rock

24,0,Rock

26,1,Hiphop

26,0,Hiphop

28,1,Classic

28,0,Classic

30,1,Pop

30,0,Pop

32,1,Dance

32,0,Dance

34,1,Rock

34,0,Rock

36,1,Hiphop

36,0,Hiphop

38,1,Classic

38,0,Classic

40,1,Pop

40,0,Pop

FOR OBJECTIVE 1 AND 2

TO VISUALIZE THE DISTRIBUTION OF MUSIC GENRES ACROSS DIFFERENT AGES AND GENDERS

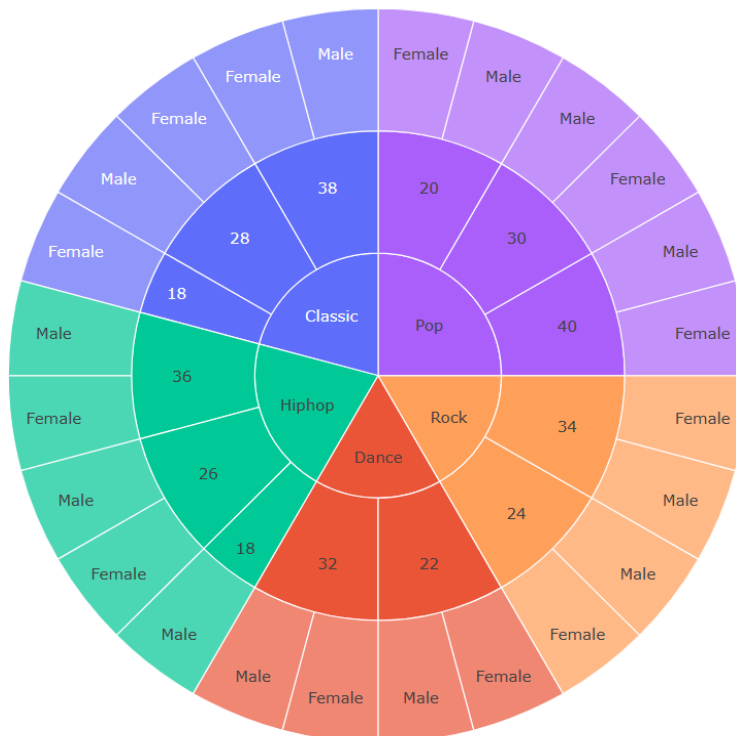
Requirements:

- Python Libraries: pandas plotly
- Files: music.csv (Dataset containing columns: genre, age, gender)

Code:

```
import pandas as pd
import plotly.express as px
df = pd.read_csv('music.csv')
df['gender_label'] = df['gender'].map({0: 'Female', 1: 'Male'})
fig = px.sunburst(
    df,
    path=['genre', 'age', 'gender_label'],
    color='genre'
)
fig.show()
```

Output:



TO DEVELOP A GENETIC ALGORITHM THAT SOLVES THE KNAPSACK PROBLEM

REQUIREMENTS

- Python 3.x
- Built-in random library

Code:

```
import random
```

```
items = [  
    {'name': 'A', 'weight': 5, 'value': 12},  
    {'name': 'B', 'weight': 3, 'value': 5},  
    {'name': 'C', 'weight': 7, 'value': 10},  
    {'name': 'D', 'weight': 2, 'value': 7}  
]  
max_weight = 12  
population_size = 20  
generations = 50  
mutation_rate = 0.1  
  
def create_individual():  
    return [random.randint(0, 1) for _ in items]  
  
def fitness(individual):  
    total_weight = sum(ind * item['weight'] for ind, item in zip(individual, items))  
    total_value = sum(ind * item['value'] for ind, item in zip(individual, items))  
    return total_value if total_weight <= max_weight else 0  
  
def select(population):  
    max_fit = sum(fitness(ind) for ind in population)  
    pick = random.uniform(0, max_fit)  
    current = 0  
    for ind in population:  
        current += fitness(ind)  
        if current >= pick:  
            return ind  
  
def crossover(parent1, parent2):  
    point = random.randint(1, len(items)-1)  
    return parent1[:point] + parent2[point:], parent2[:point] + parent1[point:]  
  
def mutate(individual):  
    for i in range(len(individual)):  
        if random.random() < mutation_rate:  
            individual[i] = 1 - individual[i]  
  
population = [create_individual() for _ in range(population_size)]  
  
for gen in range(generations):  
    new_population = []  
    for _ in range(population_size // 2):  
        parent1 = select(population)
```

```

parent2 = select(population)
child1, child2 = crossover(parent1, parent2)
mutate(child1)
mutate(child2)
new_population.extend([child1, child2])
population = new_population

best = max(population, key=fitness)
best_value = fitness(best)
best_weight = sum(ind * item['weight'] for ind, item in zip(best, items))
best_items = [item['name'] for ind, item in zip(best, items) if ind == 1]

print(f"Best value: ${best_value}")
print(f"Total weight: {best_weight} kg")
print(f"Items selected: {best_items}")

```

Output:

```

● (venv) PS E:\Normal Prog\Programming\python\4sem_ai> python -u "e:\Normal Prog\Programming\python\4sem_ai\knabsack.py"
Best value: $24
Total weight: 10 kg
Items selected: ['A', 'B', 'D']
○ (venv) PS E:\Normal Prog\Programming\python\4sem_ai> 

```

TO BUILD AND TRAIN SIMPLE ARTIFICIAL NEURAL NETWORK MODELS TO LEARN THE LOGIC OF AND AND OR GATES

REQUIREMENTS

- Python 3.x
- TensorFlow (including Keras)
- NumPy

Code:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
```

```
activation_func = 'relu'
sigmoid_func = 'sigmoid'
optimizer_algo = 'adam'
loss_algo = 'binary_crossentropy'
epochs = 500
```

```
X = np.array([[0,0], [0,1], [1,0], [1,1]])
Y_and = np.array([0, 0, 0, 1])
Y_or = np.array([0, 1, 1, 1])
```

```
def create_model():
    model = Sequential([
        Dense(4, input_dim=2, activation=activation_func),
        Dense(4, activation=activation_func),
        Dense(1, activation=sigmoid_func)
    ])
    model.compile(optimizer=optimizer_algo, loss=loss_algo, metrics=['accuracy'])
    return model
```

```
model_and = create_model()
model_and.fit(X, Y_and, epochs=epochs, verbose=0)
model_or = create_model()
model_or.fit(X, Y_or, epochs=epochs, verbose=0)
_, acc_and = model_and.evaluate(X, Y_and, verbose=0)
pred_and = np.round(model_and.predict(X, verbose=0))
_, acc_or = model_or.evaluate(X, Y_or, verbose=0)
```

```
pred_or = np.round(model_or.predict(X, verbose=0))
print(f"\nAND Model Accuracy: {acc_and:.2f}")
print("AND Predictions:\n", pred_and)
print(f"\nOR Model Accuracy: {acc_or:.2f}")
print("OR Predictions:\n", pred_or)
```

Output:

AND Model Accuracy: 1.00

AND Predictions:

[[0.]

[0.]

[0.]

[1.]]

OR Model Accuracy: 1.00

OR Predictions:

[[0.]

[1.]

[1.]

[1.]]

PS E:\Normal Prog\Programming\python\4sem_ai>

TO BUILD A TEXT CLASSIFICATION MODEL USING NAIVE BAYES ALGORITHM THAT CAN PREDICT WHETHER A GIVEN SENTENCE HAS A POSITIVE ("YES") OR NEGATIVE ("NO") SENTIMENT BASED ON TRAINING DATA

Requirements

- Python 3.x
- Scikit-learn library
- Dataset of labeled text sentences for training

Code:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

```
texts = [
```

```
    "I love this",
    "This is great",
    "I hate this",
    "This is bad"
```

```
]
```

```
labels = ["yes", "yes", "no", "no"]
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(texts)
```

```
model = MultinomialNB()
```

```
model.fit(X, labels)
```

```
text_sent = ["I love this"]
```

```
X_test = vectorizer.transform(text_sent)
```

```
pred = model.predict(X_test)
```

```
print("Sentence:", text_sent[0])
```

```
print("Prediction:", pred[0])
```

Output:

```
• (venv) PS E:\Normal Prog\Programming\python\4sem_ai> python text_classify.py
Sentence: I love this
Prediction: yes
> (venv) PS E:\Normal Prog\Programming\python\4sem_ai> 
```

TO DESIGN AND IMPLEMENT AN EXPERT SYSTEM THAT INTERACTIVELY COLLECTS USER SYMPTOMS AND PROVIDES A DIAGNOSIS BASED ON PREDEFINED RULES.

Requirements

- Prolog interpreter (e.g., SWI-Prolog)
- Defined knowledge base of symptoms and rules
- User interface for interactive questioning (console/terminal)

Code:

```
symptom(fever).
symptom(cough).
symptom(tiredness).
symptom(loss_of_taste_or_smell).
symptom(difficulty_breathing).
symptom(sore_throat).

ask_all([]).
ask_all([Symptom|Rest]) :-
    write('Do you have '), write(Symptom), write('? (yes/no): '),
    read(Response),
    ( Response == yes -> assert(yes(Symptom)) ; true ),
    ask_all(Rest).

has(Symptom) :- yes(Symptom).

has_all_yes([]).
has_all_yes([Symptom|Rest]) :-
    has(Symptom),
    has_all_yes(Rest).

has_covid :-
    findall(S, symptom(S), Symptoms),
    ( has_all_yes(Symptoms) ->
        write('You may have COVID-19. Please get tested.')
    ;
        write('You unlikely to have COVID-19.')
    ),
    nl.

start :-
    retractall(yes(_)),
    findall(S, symptom(S), Symptoms),
    write('Symptoms to check: '), write(Symptoms), nl,
    ask_all(Symptoms),
    has_covid.
```

Output:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 8.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
```

```
For online help and background, visit http://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(word).
```

```
1 ?- [covid]
```

```
.
true.
```

```
2 ?- start.
```

```
Symptoms to check: [fever,cough,tiredness,loss_of_taste_or_smell,difficulty_breathing,sore_throat]
```

```
Do you have fever? (yes/no): yes.
```

```
Do you have cough? (yes/no): |: yes.
```

```
Do you have tiredness? (yes/no): |: yes.
```

```
Do you have loss_of_taste_or_smell? (yes/no): |: yes.
```

```
Do you have difficulty_breathing? (yes/no): |: yes.
```

```
Do you have sore_throat? (yes/no): |: yes.
```

```
You may have COVID-19. Please get tested.
```

```
true.
```

TO COUNT N-GRAM FREQUENCIES (BIGRAMS) FROM A GIVEN TEXT USING NLP.

Requirements:

- Python 3.x
- Libraries: nltk ,collections (built-in)

Code:

```
import nltk
from nltk.tokenize import word_tokenize
from collections import Counter

nltk.download('punkt_tab')
def generate_ngrams(text, n):
    words = word_tokenize(text.lower()) # Smarter tokenization
    return [tuple(words[i:i+n]) for i in range(len(words) - n + 1)]

def count_ngrams(text, n):
    ngrams = generate_ngrams(text, n)
    return Counter(ngrams)

text = "Hello, this is a simple example. Simple, isn't it?"
n = 2 # Bigram
ngram_counts = count_ngrams(text, n)
for ngram, count in ngram_counts.items():
    print(f"{ngram}: {count}")
```

Output:

```
(venv) PS E:\Normal Prog\Programming\python\4sem_ai>
(venv) PS E:\Normal Prog\Programming\python\4sem_ai> python ngram.py
[nltk_data] Downloading package punkt_tab to E:\Normal
[nltk_data] Prog\Programming\python\4sem_ai\venv\nltk_data...
[nltk_data] Unzipping tokenizers\punkt_tab.zip.
('hello', ','): 1
(',', 'this'): 1
('this', 'is'): 1
('is', 'a'): 1
('a', 'simple'): 1
('simple', 'example'): 1
('example', '.'): 1
('.', 'simple'): 1
('simple', ','): 1
(',', 'is'): 1
('is', "n't"): 1
("n't", 'it'): 1
('it', '?'): 1
(venv) PS E:\Normal Prog\Programming\python\4sem_ai> □
```

TO PARSE A SENTENCE USING A CONTEXT-FREE GRAMMAR (CFG) AND DISPLAY ITS PARSE TREE.

Requirements:

- Python 3.x
- Libraries: nltk ,collections

Code:

```
import nltk
from nltk import CFG
nltk.download('punkt_tab')
grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N | Det Adj N | 'I'
VP -> V NP | V
Det -> 'a' | 'the'
N -> 'dog' | 'cat' | 'park'
Adj -> 'big' | 'small'
V -> 'saw' | 'walked'
""")
parser = nltk.ChartParser(grammar)

sentence = "I saw a big dog"
tokens = nltk.word_tokenize(sentence)
for tree in parser.parse(tokens):
    tree.pretty_print()
    tree.draw()
```

Output

